

Lab Assignment – 3

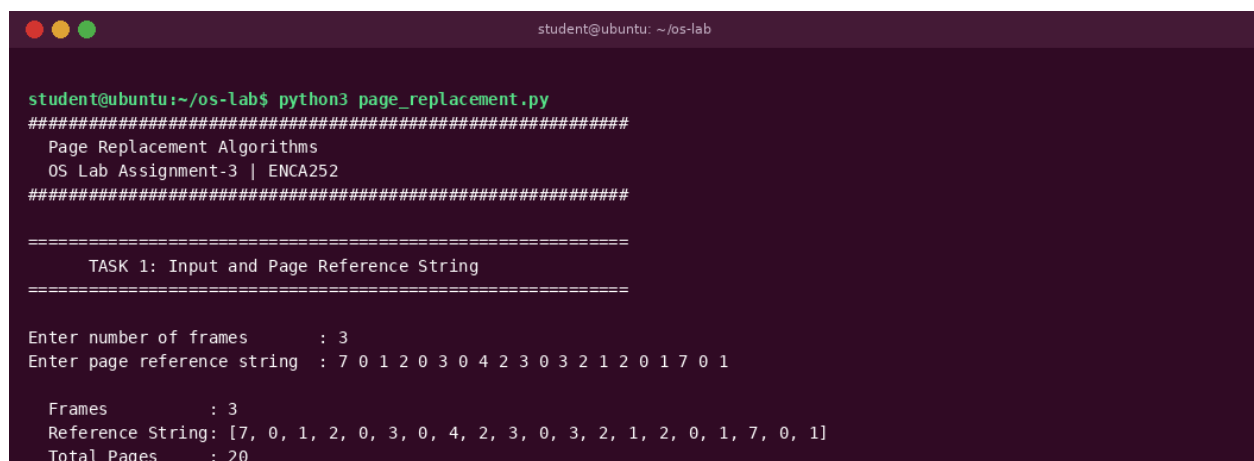
Comprehensive Implementation of Page Replacement Algorithms

Course: Fundamentals of Operating System Lab (ENCA252)
Program: BCA (AI & DS) (Research)

Task 1: Input and Page Reference String

Number of frames (3) and page reference string of 20 pages are taken as input and displayed for verification.

```
def get_input():
    frames = int(input("Enter number of frames : "))
    ref_str = list(map(int,
        input("Enter page reference string : ").split()))
    print(f" Frames : {frames}")
    print(f" Reference String: {ref_str}")
    print(f" Total Pages : {len(ref_str)}")
    return frames, ref_str
```



```
student@ubuntu: ~/os-lab

student@ubuntu:~/os-lab$ python3 page_replacement.py
#####
Page Replacement Algorithms
OS Lab Assignment-3 | ENCA252
#####

=====
TASK 1: Input and Page Reference String
=====

Enter number of frames      : 3
Enter page reference string : 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Frames      : 3
Reference String: [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1]
Total Pages : 20
```

Task 2: FIFO – First In First Out

Oldest page in memory is replaced first. A queue tracks insertion order. Page fault occurs when a requested page is not in current frames.

```
def fifo(frames, ref_str):
    memory = [] # pages currently in frame
```

```

faults = 0
queue = [] # insertion order tracker
for page in ref_str:
    fault = False
    if page not in memory:
        fault = True
        faults += 1
    if len(memory) < frames:
        memory.append(page)
        queue.append(page)
    else:
        evicted = queue.pop(0) # remove oldest page
        memory[memory.index(evicted)] = page
        queue.append(page)
print(f" Total Page Faults (FIFO) : {faults}")
return faults

```

```

student@ubuntu: ~/os-lab
student@ubuntu:~/os-lab$ python3 page_replacement.py
=====
TASK 2: FIFO Page Replacement
=====

```

Page	Frames	Fault
7	[7]	YES
0	[7, 0]	YES
1	[7, 0, 1]	YES
2	[2, 0, 1]	YES
0	[2, 0, 1]	-
3	[2, 3, 1]	YES
0	[2, 3, 0]	YES
4	[4, 3, 0]	YES
2	[4, 2, 0]	YES
3	[4, 2, 3]	YES
0	[0, 2, 3]	YES
3	[0, 2, 3]	-
2	[0, 2, 3]	-
1	[0, 1, 3]	YES
2	[0, 1, 2]	YES
0	[0, 1, 2]	-
1	[0, 1, 2]	-
7	[7, 1, 2]	YES
0	[7, 0, 2]	YES
1	[7, 0, 1]	YES

```

-----
Total Page Faults (FIFO) : 15

```

Task 3: LRU – Least Recently Used

Page that was used least recently is replaced. Past access history is scanned to find the LRU page among current frames.

```

def lru(frames, ref_str):
    memory = []
    faults = 0
    for i, page in enumerate(ref_str):
        fault = False

```

```

if page not in memory:
    fault = True
    faults += 1
    if len(memory) < frames:
        memory.append(page)
    else:
        # find page used least recently
        lru_page, lru_time = None, float("inf")
        for p in memory:
            last = max((j for j in range(i) if ref_str[j]==p),
                        default=-1)
            if last < lru_time:
                lru_time, lru_page = last, p
        memory[memory.index(lru_page)] = page
    else:
        memory.remove(page)
        memory.append(page) # mark as recently used
print(f" Total Page Faults (LRU) : {faults}")
return faults

```

```

student@ubuntu: ~/os-lab
student@ubuntu:~/os-lab$ python3 page_replacement.py
=====
TASK 3: LRU Page Replacement
=====

```

Page	Frames	Fault
7	[7]	YES
0	[7, 0]	YES
1	[7, 0, 1]	YES
2	[2, 0, 1]	YES
0	[2, 1, 0]	-
3	[2, 3, 0]	YES
0	[2, 3, 0]	-
4	[4, 3, 0]	YES
2	[4, 2, 0]	YES
3	[4, 2, 3]	YES
0	[0, 2, 3]	YES
3	[0, 2, 3]	-
2	[0, 3, 2]	-
1	[1, 3, 2]	YES
2	[1, 3, 2]	-
0	[1, 0, 2]	YES
1	[0, 2, 1]	-
7	[0, 7, 1]	YES
0	[7, 1, 0]	-
1	[7, 0, 1]	-

```

-----
Total Page Faults (LRU) : 12

```

Task 4: Optimal Page Replacement

Page that will not be used for the longest time in the future is replaced. Requires look-ahead in reference string — theoretical benchmark only.

```

def optimal(frames, ref_str):
    memory = []
    faults = 0

```

```

for i, page in enumerate(ref_str):
    if page not in memory:
        faults += 1
    if len(memory) < frames:
        memory.append(page)
    else:
        # find page used farthest in future
        farthest, evict = -1, memory[0]
        for p in memory:
            future = [j for j in range(i+1, len(ref_str))
                      if ref_str[j] == p]
            next_use = future[0] if future else float("inf")
            if next_use > farthest:
                farthest, evict = next_use, p
        memory[memory.index(evict)] = page
print(f" Total Page Faults (OPT) : {faults}")
return faults

```

```

student@ubuntu: ~/os-lab
student@ubuntu:~/os-lab$ python3 page_replacement.py
=====
TASK 4: Optimal Page Replacement
=====

```

Page	Frames	Fault
7	[7]	YES
0	[7, 0]	YES
1	[7, 0, 1]	YES
2	[2, 0, 1]	YES
0	[2, 0, 1]	-
3	[2, 0, 3]	YES
0	[2, 0, 3]	-
4	[2, 4, 3]	YES
2	[2, 4, 3]	-
3	[2, 4, 3]	-
0	[2, 0, 3]	YES
3	[2, 0, 3]	-
2	[2, 0, 3]	-
1	[2, 0, 1]	YES
2	[2, 0, 1]	-
0	[2, 0, 1]	-
1	[2, 0, 1]	-
7	[7, 0, 1]	YES
0	[7, 0, 1]	-
1	[7, 0, 1]	-

```

-----
Total Page Faults (OPT) : 9

```

Task 5: MRU – Most Recently Used

Most recently used page is evicted first. Useful in specific cyclic workloads where recently used pages are unlikely to be needed again soon.

```

def mru(frames, ref_str):
    memory = []
    faults = 0
    recent = None # track most recently used page
    for page in ref_str:

```

```

if page not in memory:
    faults += 1
if len(memory) < frames:
    memory.append(page)
else:
    # evict the most recently used page
    if recent in memory:
        memory[memory.index(recent)] = page
    else:
        memory[-1] = page
    recent = page # update most recently used
print(f" Total Page Faults (MRU) : {faults}")
return faults

```

```

student@ubuntu: ~/os-lab
student@ubuntu:~/os-lab$ python3 page_replacement.py
=====
TASK 5: MRU Page Replacement
=====

```

Page	Frames	Fault
7	[7]	YES
0	[7, 0]	YES
1	[7, 0, 1]	YES
2	[7, 0, 2]	YES
0	[7, 0, 2]	-
3	[7, 3, 2]	YES
0	[7, 0, 2]	YES
4	[7, 4, 2]	YES
2	[7, 4, 2]	-
3	[7, 4, 3]	YES
0	[7, 4, 0]	YES
3	[7, 4, 3]	YES
2	[7, 4, 2]	YES
1	[7, 4, 1]	YES
2	[7, 4, 2]	YES
0	[7, 4, 0]	YES
1	[7, 4, 1]	YES
7	[7, 4, 1]	-
0	[0, 4, 1]	YES
1	[0, 4, 1]	-

```

-----
Total Page Faults (MRU) : 16

```

Task 6: Performance Comparison

All four algorithms are ranked by total page faults. Optimal scores best (9), followed by LRU (12), FIFO (15), and MRU (16).

```

def performance_comparison(fifo_f, lru_f, opt_f, mru_f):
    results = [
        ("FIFO", fifo_f),
        ("LRU", lru_f),
        ("Optimal", opt_f),
        ("MRU", mru_f),
    ]
    results_sorted = sorted(results, key=lambda x: x[1])
    print(f" Algorithm Page Faults Rank")

```

```

for rank, (algo, faults) in enumerate(results_sorted, 1):
print(f" {algo:<12} {faults:<14} {rank}")
print(f" Best : {results_sorted[0][0]} ({results_sorted[0][1]} faults)")
print(f" Worst : {results_sorted[-1][0]} ({results_sorted[-1][1]} faults)")

```

```

student@ubuntu: ~/os-lab$ python3 page_replacement.py
=====
TASK 6: Performance Comparison
=====

Algorithm    Page Faults    Rank
-----
Optimal      9              1
LRU          12             2
FIFO         15             3
MRU          16             4
-----

Best Algorithm : Optimal (9 faults)
Worst Algorithm : MRU (16 faults)

```

Task 7: Result Analysis

Optimal is the theoretical minimum but requires future knowledge. LRU is the best practical choice. FIFO is simple but suffers Belady's Anomaly. MRU suits cyclic patterns but performs worst here.

```

def result_analysis(fifo_f, lru_f, opt_f, mru_f):
print(f"""
+-----+-----+-----+
| Algorithm | Page Faults | Key Property |
+-----+-----+-----+
| FIFO | {fifo_f:<12}| Simplest; ignores usage pattern |
| LRU | {lru_f:<12}| Temporal locality; near-optimal |
| Optimal | {opt_f:<12}| Theoretical minimum; future known|
| MRU | {mru_f:<12}| Best for cyclic access patterns |
+-----+-----+-----+

- Optimal gives minimum page faults (theoretical benchmark).
- LRU closely approaches Optimal using past access history.
- FIFO is simple but suffers from Belady Anomaly.
- MRU is poor for sequential but good for cyclic workloads.
Conclusion: LRU is best for real-world OS implementation.
""")

```

student@ubuntu:~/os-lab\$ python3 page_replacement.py

```
=====
TASK 7: Result Analysis
=====
```

Summary:

Algorithm	Page Faults	Key Property
FIFO	15	Simplest; ignores usage pattern
LRU	12	Temporal locality; near-optimal
Optimal	9	Theoretical minimum; future known
MRU	16	Best for cyclic access patterns

Analysis:

- Optimal gives the MINIMUM page faults (theoretical benchmark).
- LRU closely approaches Optimal using past access history.
- FIFO is simple but suffers from Belady's Anomaly.
- MRU performs poorly for sequential access but can excel in specific cyclic or looping workloads.

Conclusion:

LRU is the most practical algorithm for real-world OS use.
 It approximates Optimal without requiring future knowledge.
 FIFO is used when simplicity is preferred over efficiency.
 Optimal serves as a benchmark - not implementable in practice.

student@ubuntu:~/os-lab\$